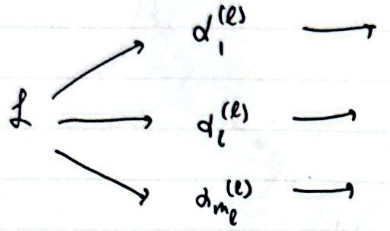
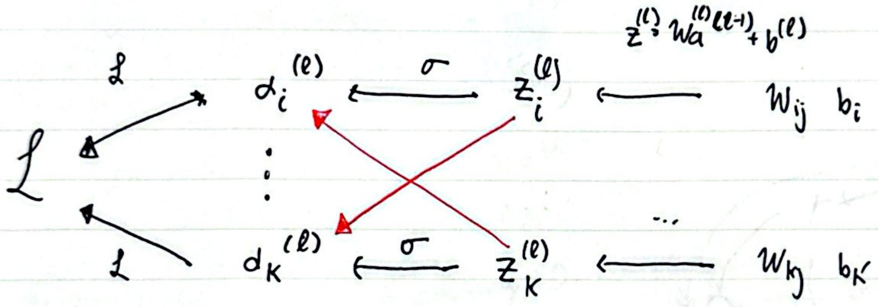
$$\frac{\partial L}{\partial p} = \sum_{c=1}^{m_\ell} \frac{\partial L}{\partial a_c^{(\ell)}} \frac{\partial a_c^{(\ell)}}{\partial p}$$

$$= \sum_{c=1}^{m_\ell} \frac{\partial L}{\partial a_c^{(\ell)}} \cdot \frac{\partial a_c^{(\ell)}}{\partial z_c^{(\ell)}} \cdot \frac{\partial z_c^{(\ell)}}{\partial p}$$

der L wrt ~~loss~~ activation value — der activation pre-activation — der. of output wrt. weight.

$\frac{\partial L}{\partial a_c^{(\ell)}}$ → calculated recursively.





before ~~...~~. $$\frac{\partial L}{\partial W_{ij}^{(\ell)}} = \frac{\partial L}{\partial a_i^{(\ell)}} \cdot \frac{\partial a_i^{(\ell)}}{\partial z_i^{(\ell)}} \cdot \frac{\partial z^{(\ell)}}{\partial W_{ij}^{(\ell)}}$$

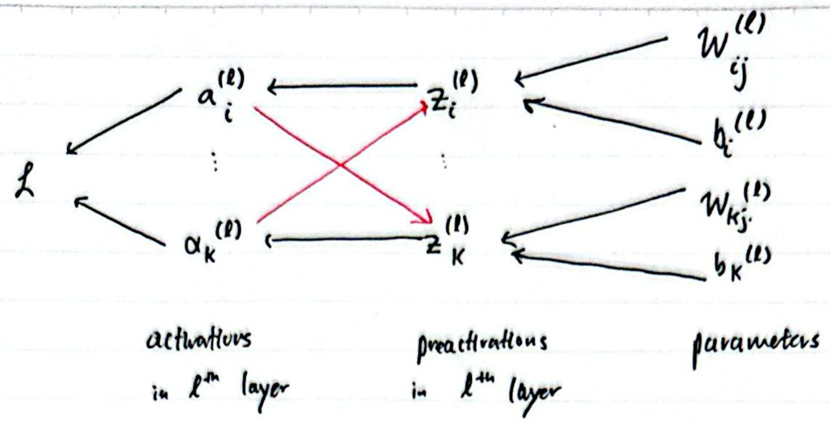
⇒ now: $$\frac{\partial L}{\partial W_{ij}} = \frac{\partial L}{\partial a_i^{(\ell)}} \cdot \frac{\partial a_i^{(\ell)}}{\partial z_i^{(\ell)}} \cdot \frac{\partial z_i^{(\ell)}}{\partial W_{ij}^{(\ell)}} + \frac{\partial L}{\partial a_K^{(\ell)}} \cdot \frac{\partial a_K^{(\ell)}}{\partial z_i^{(\ell)}} \cdot \frac{\partial z_i}{\partial W_{ij}^{(\ell)}}$$

same

Σ.

diffrent...

apply_derivative needs two ~~...~~ parameters,
output_idx
input_idx



→ interdependence.

before: $\dfrac{\partial \mathcal{L}}{\partial p_i} = \dfrac{\partial \mathcal{L}}{\partial a_i^{(l)}} \cdot \dfrac{\partial a_i^{(l)}}{\partial z_i^{(l)}} \cdot \dfrac{\partial z_i^{(l)}}{\partial p_i}$ where $p_i \in \{ b_i^{(l)}, W_{ij}^{(l)} \}$

this was true where our activation functions are non interdependent, ie $a_i^{(l)}$ does not depend on $z_k^{(l)}$ $i \neq k$.

but for interdependent activation functions, $a_k^{(l)}$ depends on $z_i^{(l)}$ (see diagram)

thus: $\dfrac{\partial \mathcal{L}}{\partial p_i} = \dfrac{\partial \mathcal{L}}{\partial a_i^{(l)}} \cdot \dfrac{\partial a_i^{(l)}}{\partial z_i^{(l)}} \cdot \dfrac{\partial z_i^{(l)}}{\partial p_i}$

$$+ \dfrac{\partial \mathcal{L}}{\partial a_k^{(l)}} \cdot \underbrace{\dfrac{\partial a_k^{(l)}}{\partial z_i^{(l)}}}_{\text{previously } 0 \text{ if non interdependence}} \cdot \dfrac{\partial z_i^{(l)}}{\partial p_i}$$

⇒ generalize: $\dfrac{\partial \mathcal{L}}{\partial p_i} = \left(\sum_{c=1}^{m_L} \dfrac{\partial \mathcal{L}}{\partial a_c^{(l)}} \cdot \dfrac{\partial a_c^{(l)}}{\partial z_i^{(l)}} \right) \cdot \dfrac{\partial z_i^{(l)}}{\partial p_i}$

sum over all layer dimensions

recall
$p_i = W_{ij}^{(l)}$ ⇒ $a_j^{(l-1)}$
$p_i = b_i$ ⇒ 1

implement: network.cpp
curr_gradient still stores $\begin{bmatrix} \dfrac{\partial \mathcal{L}}{\partial a_c^{(l)}} \end{bmatrix}_{c=1}^{m_L}$ as result.

given i: $\underbrace{\text{imed\_value}}_{\text{change!}}$ = $\underset{c=0 \to m_L-1}{\text{sum}}$ { curr_gradient [c] * jacobian ($z^{(l)}$, c, i) }